

PostgreSQL CDC with Debezium

Debezium captures row-level changes in PostgreSQL by reading the write-ahead log (WAL) through PostgreSQL's logical decoding feature, not the table directly. A logical decoding *output plug-in* on the server side turns WAL entries into a consumable stream; Debezium's Java connector reads that stream over the standard replication protocol and emits Kafka events. Unlike Mongo (where the driver already gives you **before/after** as embedded JSON strings), Postgres logical decoding gives you actual structured row data per plug-in.

Two plug-in choices:

- **pgoutput** — built into Postgres 10+, no extra install, maintained by the Postgres community itself (it's the same thing native logical replication uses). Default choice today.
- **decoderbufs** — Debezium-maintained, Protobuf-based, requires installing a separate extension. Mostly legacy at this point; use only if you're stuck on very old Postgres or have a specific reason.

This doc assumes **pgoutput**.

1. PostgreSQL side — admin setup runbook

Nine steps, in order. Each one says why it exists and gives a query to confirm it actually worked before moving to the next — don't skip the verification lines, since half of these fail silently until a connector tries to attach and you get a confusing error three steps removed from the actual cause.

1.1 Choose a plug-in

- **pgoutput** — built into Postgres 10+, no install, Postgres-maintained. Use this unless you have a specific reason not to; it's the only option on every managed Postgres (RDS, Cloud SQL, Azure, CrunchyBridge) anyway.
- **decoderbufs** — Debezium-maintained, Protobuf-based, requires installing a separate extension. Legacy at this point.

This doc uses **pgoutput** from here on. Nothing to run for this step — it's a decision that shows up later as **plugin.name**.

1.2 Enable logical decoding

In **postgresql.conf** (requires a full server restart, not just a reload):

```
wal_level = logical
max_wal_senders = 4
max_replication_slots = 4
```

- **wal_level = logical** — WAL entries include enough info to reconstruct row changes, not just physical replication. Mandatory; nothing works without it.
- **max_wal_senders** — number of concurrent processes allowed to stream WAL out. One per connector/slot in use, plus headroom.

- `max_replication_slots` — cap on how many slots can exist at once. Each Debezium connector owns exactly one slot.

Optional but worth knowing: `wal_keep_size` caps how much WAL a slot can force the server to retain before things get ugly (see the disk-bloat note later), and `wal_writer_delay` (default 200ms) adds latency if `synchronous_commit` isn't `on` — drop it to something like 10ms if you care about event lag.

Verify:

```
SHOW wal_level; SHOW max_wal_senders; SHOW max_replication_slots;
```

On managed Postgres you usually can't touch `postgresql.conf` directly — there's an instance-level flag instead, and it still needs a restart to take effect:

- **RDS:** set `rds.logical_replication = 1` on the parameter group, restart, then `SHOW wal_level` as the master user. Grant `rds_replication` to any non-master account that needs to run the connector or take the initial snapshot.
- **Cloud SQL:** set the `cloudsql.logical_decoding` flag to `on` — this auto-flips `wal_level` to `logical` for you.
- **Azure Database for PostgreSQL:** `az postgres server configuration set --name azure.replication_support --value logical`, then restart.

All three (plus CrunchyBridge) force `plugin.name: pgoutput` — no ability to install `decoderbufs`. Set it explicitly rather than relying on a default.

1.3 Create a replication user

```
CREATE ROLE MDB_CDC WITH LOGIN PASSWORD '<password>' REPLICATION;
```

`REPLICATION` is the role attribute that actually lets the user open a replication connection — without it Postgres refuses the connector outright, no matter what table grants you hand out. Best practice (and the point of this whole step) is a scoped role, not the instance superuser.

Verify:

```
SELECT rolname, rolreplication, rolcanlogin FROM pg_roles WHERE rolname = 'MDB_CDC';
```

Both flags should read `t`.

1.4 Grant what's needed to create a publication

To add a table to a publication (next step) the user must be its *owner* — and your Debezium user almost never owns the application's tables. Rather than handing Debezium superuser or reassigning ownership

outright, share ownership through a group role, then grant **CREATE** on the database for the publication object itself:

```
CREATE ROLE replication_group;
GRANT replication_group TO <original_table_owner>;
GRANT replication_group TO MDB_CDC;
ALTER TABLE mydb.orders OWNER TO replication_group;
GRANT CREATE ON DATABASE mydb TO MDB_CDC;
```

A role inherits the privileges of any group it belongs to, so **MDB_CDC** now behaves as an owner of **orders** — including the **SELECT** it needs for the initial snapshot — without literally being the owner, and the original owner doesn't lose access either.

Verify:

```
SELECT tableowner FROM pg_tables WHERE tablename = 'orders';
SELECT has_database_privilege('MDB_CDC', 'mydb', 'CREATE');
SELECT has_table_privilege('MDB_CDC', 'mydb.orders', 'SELECT');
```

Expect **replication_group** as owner and both privilege checks to return **t**.

1.5 Create the publication

pgoutput streams changes for tables that belong to a *publication* — a Postgres-native logical-replication concept with no Mongo equivalent. It's best practice to create this yourself ahead of time rather than lean on Debezium's autcreate:

```
CREATE PUBLICATION dbz_publication FOR TABLE mydb.orders, mydb.customers;
```

Match this to the connector's **publication.name** and **table.include.list** — if a table isn't in the publication, Debezium never sees its changes even if it's listed in **table.include.list**.

Verify:

```
SELECT pubname, puballtables FROM pg_publication;
SELECT * FROM pg_publication_tables;
```

puballtables should read **f** (scoped, not all-tables), and both target tables should show up in **pg_publication_tables**.

If Debezium is left to autcreate instead, set **publication.autocreate.mode: filtered** in the connector config — the default (**all_tables**) tries **CREATE PUBLICATION ... FOR ALL TABLES**, which fails outright if the user lacks ownership on every table in the database (the default failure mode on CrunchyBridge specifically, so set **filtered** there as a matter of course).

1.6 Create the replication slot

The slot is what guarantees no data loss — Postgres retains every WAL segment the slot hasn't confirmed as processed, even across a Debezium outage. That guarantee cuts both ways: an orphaned slot (connector gone, slot still there) silently accumulates WAL and eats disk forever, so name it deliberately and know what it's for.

```
SELECT pg_create_logical_replication_slot('dbz_orders_slot', 'pgoutput');
```

Match `dbz_orders_slot` to the connector's `slot.name`. Each connector needs its own uniquely-named slot — sharing one across connectors doesn't work.

Verify:

```
SELECT slot_name, plugin, slot_type, active FROM pg_replication_slots;
```

Expect the new row with `active = f` — it only flips to `t` once a connector actually attaches.

Manual creation (over letting Debezium auto-create on first connect) is the right call for: production generally, a controlled Postgres version upgrade where you need to preserve change data across it, failover recovery where the slot needs to exist on the new primary before writes resume, or a connector user that intentionally lacks `REPLICATION/CREATE`.

1.7 Allow the replication connection

Postgres authorizes replication-protocol connections separately from ordinary SQL connections, even for the same user. In `pg_hba.conf`, add an entry for the connector's host:

```
host      replication      MDB_CDC      <connector-host-ip>/32      scram-sha-256
```

then reload (`SELECT pg_reload_conf();` — no restart needed for `pg_hba.conf` changes).

Verify — do this before ever touching Kafka Connect, since it isolates auth from everything else:

```
psql "host=<postgres-host> dbname=mydb replication=database user=MDB_CDC" -  
c "IDENTIFY_SYSTEM;"
```

If `pg_hba.conf` is wrong, this fails immediately with `no pg_hba.conf entry for replication connection`. If it's right, you get back a row with the system identifier and current WAL location — proof the auth path works independent of Debezium.

1.8 Replica identity — the important gotcha

By default (`REPLICA IDENTITY DEFAULT`), `UPDATE/DELETE` events only carry the **primary key** columns in `before` — not the full old row. If downstream needs the complete previous row state — the direct equivalent of Mongo's `capture.mode: change_streams_update_full` — set:

```
ALTER TABLE mydb.orders REPLICA IDENTITY FULL;
```

Without this, don't expect `before` to help you diff old vs. new values; you'll only get the PK.

Verify:

```
SELECT relreplident FROM pg_class WHERE oid = 'mydb.orders'::regclass;
```

`f` = FULL, `d` = DEFAULT (PK-only).

1.9 Final checklist before handing off

Run all of these together as a last pass — if everything here comes back clean, the Kafka Connect side (Section 2) will register without surprises:

```
SELECT rolname, rolreplication FROM pg_roles WHERE rolname = 'MDB_CDC';
SELECT pubname, puballtables FROM pg_publication;
SELECT slot_name, plugin, active FROM pg_replication_slots;
SELECT relreplident FROM pg_class WHERE oid = 'mydb.orders'::regclass;
```

2. Install the connector on Kafka Connect

Drop the Debezium PostgreSQL connector JAR into your Connect plugin path (e.g. `<kafka-plugins-path>/debezium-connector-postgres/`), restart the Connect workers.

Register via REST API:

```
curl -X POST http://connect-host:8083/connectors \
  -H "Content-Type: application/json" \
  -d @postgres-connector.json
```

`postgres-connector.json`:

```
{
  "name": "postgres-cdc-connector",
  "config": {
    "connector.class":
      "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "<postgres-host>",
```

```

    "database.port": "5432",
    "database.user": "MDB_CDC",
    "database.password": "<password>",
    "database.dbname": "mydb",
    "topic.prefix": "dbserver1",
    "table.include.list": "mydb.orders,mydb.customers",
    "plugin.name": "pgoutput",
    "slot.name": "dbz_orders_slot",
    "publication.name": "dbz_publication",
    "publication.autocreate.mode": "filtered",
    "snapshot.mode": "initial",
    "tombstones.on.delete": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter.schemas.enable": "false"
  }
}

```

What each parameter controls:

- `database.hostname` / `port` / `dbname` — single primary connection, unlike Mongo's multi-host replica-set string. Postgres logical slots only live on a primary; you can't point the connector at a replica.
- `topic.prefix` — same role as in Mongo: prefix for generated topic names, but the naming pattern is `topicPrefix.schemaName.tableName` (schema-qualified), e.g. `dbserver1.mydb.orders` — one topic per table.
- `table.include.list` — whitelist, same idea as Mongo's `collection.include.list`. Fully-qualified `schema.table`.
- `plugin.name` — `pgoutput` or `decoderbufs`. Must match how the server/plugin was set up.
- `slot.name` — the replication slot this connector owns. One connector = one slot, and the slot persists on the server between connector restarts, holding WAL until Debezium confirms it's processed. **A slot with no connector reading from it is a live WAL-bloat risk** — Postgres won't reclaim WAL segments the slot hasn't confirmed past, even if the connector is gone for good.
- `publication.name` — must match (or be creatable as) the publication from step 1.5.
- `publication.autocreate.mode` — `filtered` scopes an auto-created publication to `table.include.list`; the default `all_tables` tries to grab every table in the database and fails outright if the user lacks the privilege for that.
- `snapshot.mode` — same menu of values as Mongo conceptually (`initial`, `no_data`, `never/initial_only` etc.), plus a few Postgres-only modes (`always`, `when_needed`, `configuration_based`) for things like recovering after a failover promotes a new primary.
- `tombstones.on.delete` — same as Mongo: null-value marker after deletes for Kafka log compaction, filtered out in NiFi.

Check it came up:

```
curl http://connect-host:8083/connectors/postgres-cdc-connector/status
```

3. Event shape

```
{
  "before": null,
  "after": {
    "id": 1,
    "status": "shipped",
    "amount": 250
  },
  "op": "u",
  "ts_ms": 1752480000123
}
```

Key difference from Mongo: **after** (and **before**) are already structured JSON objects, not JSON-encoded strings. No double-parsing needed in NiFi — one **EvaluateJsonPath** pass gets you there.

op values (Postgres has two more than Mongo):

- **r** — snapshot read (initial load, treat as insert)
- **c** — insert
- **u** — update
- **d** — delete
- **t** — truncate (whole table wiped; no message key, needs its own handling — see below)
- **m** — generic logical decoding message (only via **pgoutput** on Postgres 14+, rare unless something explicitly calls **pg_logical_emit_message**)

before on **u/d** events only contains what **REPLICA IDENTITY** exposes — PK columns only by default, full row if you set **REPLICA IDENTITY FULL** (see 1.8).

Also note: a primary-key **update** doesn't come through as a single **u** event — Debezium splits it into a **d** (old key) + **c** (new key), each carrying a message header (**__debezium.newkey** / **__debezium.oldkey**). If your NiFi flow assumes PK is immutable per row, this is the edge case that breaks that assumption.

Notes

- **WAL disk bloat is the #1 operational headache with Postgres CDC**, more so than with Mongo's oplog. A replication slot holds onto WAL segments until Debezium confirms it has processed past them. If the connector is down, misconfigured, or just slow, WAL piles up on the server disk — potentially filling it. Check **pg_replication_slots**: if **confirmed_flush_lsn** isn't advancing while **restart_lsn** lags behind, the server isn't reclaiming space and something downstream is stuck.
- Drop the replication slot manually (**SELECT pg_drop_replication_slot('dbz_orders_slot')**) if you're permanently decommissioning a connector — an orphaned slot doesn't clean itself up and will quietly eat disk indefinitely.
- If Connect restarts, it resumes from the last committed LSN (the offset), same no-replay guarantee as Mongo's committed Kafka offset — as long as the WAL segment covering it hasn't been discarded, which ties back to the point above.

- Postgres has a fixed schema per table (unlike Mongo), so schema drift shows up differently: logical decoding doesn't propagate DDL changes to the event stream, so adding/dropping/renaming columns won't automatically flow through — you'll need to update `table.include.list`/downstream schema handling or trigger an ad hoc (incremental/blocking) snapshot to pick up the new shape.
- Set backpressure thresholds on the queue after `ConsumeKafka`, same reasoning as Mongo — a slow sink shouldn't be allowed to pile up unbounded memory in NiFi.
- Reference: [Debezium PostgreSQL connector docs](#)